

Самое главное!!! 1. С "claude.ai" работаем так: либо: "Покажи мне файл X" — я вставляю содержимое, ты говоришь, что куда. Либо: "Вставь вот этот код в файл X, между строкой ... и строкой ..." — я иду и делаю. Либо: "В файле X найди функцию def handle_intimidate и замени её целиком вот на это." Никаких "вот общая идея" — только точные хирургические инструкции.

2. Ты просишь предоставить старый файл, который содержит логику нового файла, а потом на основе старого файла переписываешь и дописываешь новый файл.

Один огромный orchestrator.py (700+ строк) — это главная причина, почему всё ломается и почему мы уже месяц латаем симптомы вместо того, чтобы вылечить болезнь.

1. Что реально сломано (корневые причины):

№	Проблема	Почему это критично	Где именно сидит
1	Оркестратор — монолит	Вся логика в одном файле. Любое изменение ломает 5 других мест. Нет тестов, нет читаемости.	orchestrator.py (run_turn + stream_turn + _run_python_engines + _extract_player_target)
2	Два независимых классификатора	action_classifier.py и _classify в sandbox_handler.py — дублируют друг друга и оба слабые.	Два разных файла
3	SceneChange генерируются только внутри sandbox	Если действие не попало в _SANDBOX_TYPES — изменений от игрока никогда не будет.	_run_python_engines + sandbox_handler
4	Нет единого пайплайна	action → classify → scene_changes → apply → context → agents — цепочка рвётся в 3 местах.	orchestrator + sandbox + dm_agent
5	DM и NPC строят контекст по-разному	scene_state мутируется, но NPC-агент часто читает major_npcs.json напрямую.	dm_agent + npc_agent + _get_npcs_in_location
6	Психология без decay и без единого источника	Stress накапливается каждый ход, breakpoint слишком низкий, _apply_npc_state_updates вызывается не всегда.	npc/psyche_engine + orchestrator
7	Data-driven vs hardcoded	Ты хочешь generic, а сейчас 90% логики — keyword-таблицы внутри Python-файлов.	sandbox_handler, action_classifier, KEYWORDS
8	stream_turn и run_turn дублируют код	Два почти одинаковых пути — бомба.	orchestrator.py (две огромные функции)

Это не мелкие баги. Это архитектурные долги, которые делают дальнейшую разработку адом.

2. Новая архитектура (то, что я предлагаю):

Цель: Каждый файл отвечает **ровно за одно** (Single Responsibility Principle).

Оркестратор исчезает как монолит. Вместо него появляется **тонкий GameLoop**.

Enigma/

```
└─ start_enigma.bat      ← ЕДИНАЯ ТОЧКА ВХОДА
└─ reload_enigma.bat

└─ launcher.py          ← (будет) нативное окно
|
└─ backend/
|   └─ app/
|       └─ main.py      ← FastAPI + startup (запускает GameLoop)
|       |
|       | └─ api/
|       |     └─ routes.py    ← REST (run_turn)
|       |     └─ routes_stream.py ← SSE (stream_turn) — только транспорт
|       |     └─ routes_debug.py
|       |
|       | └─ services/
|       |     |
|       |     | └─ game_loop.py    ← ★★★ ЦЕНТР ВСЕГО (новый, тонкий)
|       |     |         → вызывает processor + agents
|       |     |
|       |     | └─ action/        ← Phase 1 (Core)
|       |     |     └─ processor.py ← ЕДИНЫЙ ПАЙПЛАЙН одного хода
|       |     |     └─ classifier.py ← Data-driven (читает patterns/*.yaml)
|       |     |         └─ patterns/ ← YAML с тысячами русских фраз
|       |     |             └─ physical.yaml
```

- | | | | └─ environmental.yaml
- | | | | └─ social.yaml
- | | | | └─ combat.yaml
- | | | | └─ ...
- | | | |
- | | | └─ state/ ← Phase 1 (Core) — единственный источник правды
- | | | | └─ scene_state_manager.py
- | | | | └─ scene_change.py
- | | | | └─ context_builder.py ← ЕДИНЫЙ контекст для DM + NPC
- | | | |
- | | | └─ engines/ ← Python считает (все математические движки)
- | | | | └─ combat_math.py
- | | | | └─ physics_validator.py
- | | | | └─ dnd_rules/ ← ★ НОВЫЙ МОДУЛЬ (все правила D&D 5e)
- | | | | | └─ core.py
- | | | | | └─ character_progression.py
- | | | | | └─ spell_engine.py
- | | | | | └─ rest_engine.py
- | | | | | └─ combat_turn_manager.py
- | | | | | └─ economy.py
- | | | | | └─ rules_validator.py
- | | | | └─ sandbox/
- | | | | | └─ handler_registry.py
- | | | | | └─ handlers/ ← По одному файлу на тип действия
- | | | | | | └─ environmental.py
- | | | | | | └─ physical.py
- | | | | | | └─ ...
- | | | | └─ npc_psychology/ ← 8 движков NPC

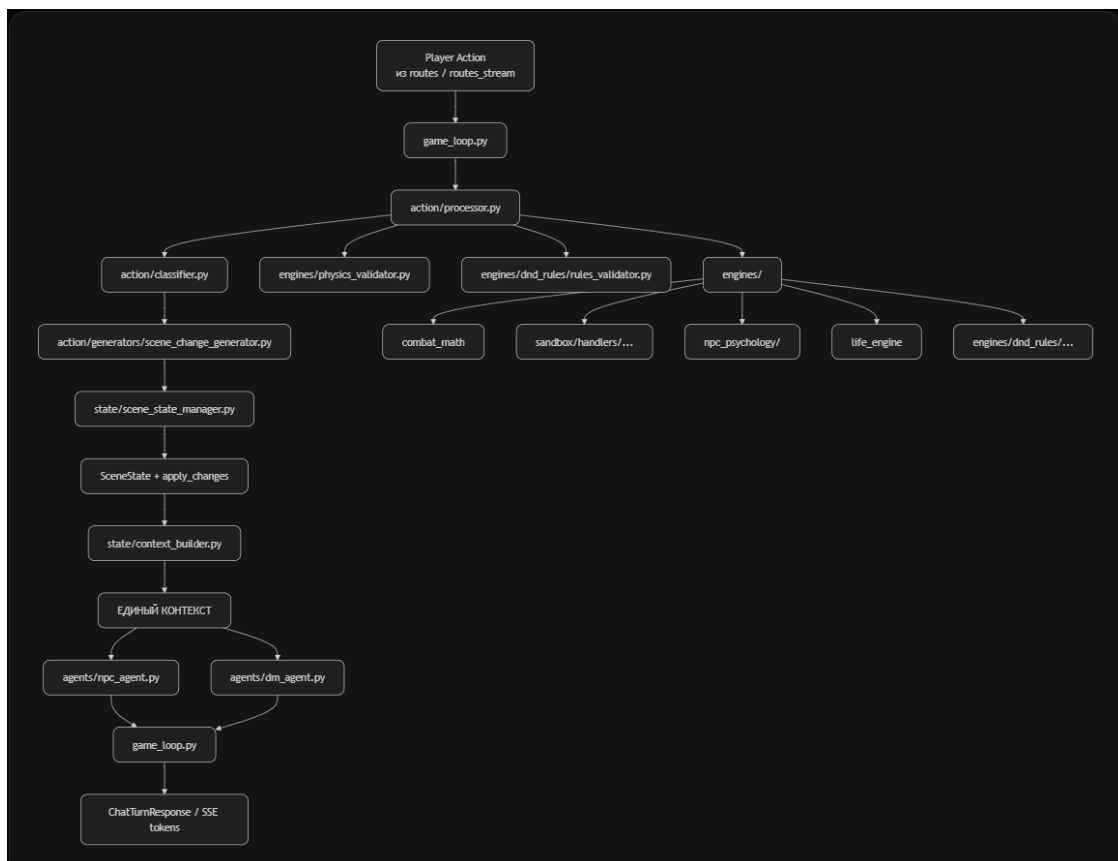
```
| | | | ├── life_engine.py
| | | | └── world_loader.py ← Phase 2 (PDF-кампании)
| | | |
| | | | ├── knowledge/      ← Phase 2 (будущее PDF)
| | | | ├── pdf_loader.py
| | | | ├── rag_index.py
| | | | └── campaign_importer.py
| | | |
| | | | ├── agents/         ← LLM-агенты (только рассказывают)
| | | | ├── dm_agent.py
| | | | ├── npc_agent.py
| | | | ├── rules_agent.py ← теперь почти не нужен
| | | | └── ...
| | | |
| | | | ├── memory/         ← LayeredMemory
| | | | └── llm/            ← router + providers
| | |
| | └── models/
| |     └── schemas.py
| |
| └── data/
|     ├── campaigns/
|     ├── npcs/
|     ├── locations/
|     ├── knowledge_db/    ← Phase 2 (после PDF)
|     └── logs/
|
└── frontend/ui/index.html
```

Я сделал её **максимально прозрачной**:

- Каждому важному файлу — чёткое описание
- Все зависимости и потоки данных (кто → кого → что передаёт)
- Ранжирование по приоритетам (Core / Phase 1 / Phase 2 / Future)
- Соответствие твоим принципам (Python считает — LLM рассказывает, SceneState — единственный источник правды, data-driven, готовность к PDF-кампаниям)

Главное изменение:

- game_loop.py — это **единственное** место, где вызываются все этапы.
- action/processor.py — новый "мозг" одного хода игрока.
- Классификация и генерация SceneChange — **data-driven** (YAML-файлы).



Что куда передаётся:

Шаг	Что передаётся	Куда	Что возвращает
1	ChatTurnRequest (actions + location)	game_loop.run_turn() / stream_turn()	—
2	actions	action/processor.py	ProcessingResult
3	text	classifier.py	ActionType + confidence
4	ActionType	scene_change_generator.py	list[SceneChange]
5	SceneChange + SceneState	scene_state_manager.apply_changes()	обновлённый SceneState
6	обновлённый SceneState + player data	все engines	результаты (combat, sandbox, psychology...)
7	все результаты	context_builder.py	единый context dict
8	context	npc_agent + dm_agent	нарратив + реакции
9	финальный результат	game_loop → routes	ответ игроку

Ранжирование по приоритетам (что делать в каком порядке)

Core / MVP (должно работать уже сейчас — 1–2 недели)

- game_loop.py
- action/processor.py
- action/classifier.py + patterns/*.yaml
- state/context_builder.py
- state/scene_state_manager.py (уже почти готов)
- engines/dnd_rules/core.py + rules_validator.py
- engines/sandbox/handlers/ (разбить текущий sandbox_handler)
-

Phase 1 (следующие 2–3 недели)

- Полная миграция из старого orchestrator.py
- engines/dnd_rules/combat_turn_manager.py (инициатива, ходы боя)
- engines/dnd_rules/character_progression.py
- engines/dnd_rules/spell_engine.py
- engines/dnd_rules/rest_engine.py
- engines/dnd_rules/economy.py

Phase 2 (после MVP)

- knowledge/world_loader.py + PDF-импорт
- knowledge/campaign_importer.py
- Мультиплеер (до 8 игроков за ход)
- engines/npc_psychology/reaction_priority.py

Future (Phase 3+)

- Полноценный RAG над PDF-книгами
- Автогенерация NPC/локаций из описания кампании
- Karma Engine, Social Mobility и т.д.

Итоговые преимущества новой структуры

1. **Один файл = одна ответственность** — больше никогда не будет 700-строчного монолита.
2. **Data-driven** — новые действия и фразы добавляются в YAML, код не трогается.
3. **D&D правила в Python** — вся математика, уровни, магия, отдых, экономика — в dnd_rules/.
4. **Один источник правды** — context_builder.py гарантирует, что DM и NPC видят одинаковый мир.
5. **Готовность к PDF** — knowledge/ уже выделен и ждёт фазу 2.
6. **Легко тестировать** — каждый модуль можно тестировать отдельно.